

Estudo da Aula 07: Expressões, Atribuição e Controle

1. Objetivo da aula

A aula trata de como as linguagens avaliam expressões, guardam valores (atribuição) e controlam a ordem de execução (estruturas de controle). A ideia é entender o que o código realmente faz, sem se enganar com precedência, conversão implícita ou desvio de fluxo.

Três casos reais mostram por que isso importa:

goto fail (Apple, 2014): uma linha `goto fail;` duplicada, logo depois de um `if` sem chaves, fazia a verificação da assinatura ser pulada. iOS e macOS aceitavam certificados falsos.

Zune (Microsoft, 2008): no último dia de um ano bissexto, um laço `while` não fazia progresso e nunca terminava. Os aparelhos travavam.

PHP e `==`: `"0e123" == "0e456"` dá verdadeiro (os dois viram 0 em notação científica), então logins que comparavam hashes com `==` aceitavam senha errada.

Nos três casos o erro era de estrutura de controle ou de avaliação de expressão, e não de regra de negócio.

2. Expressões

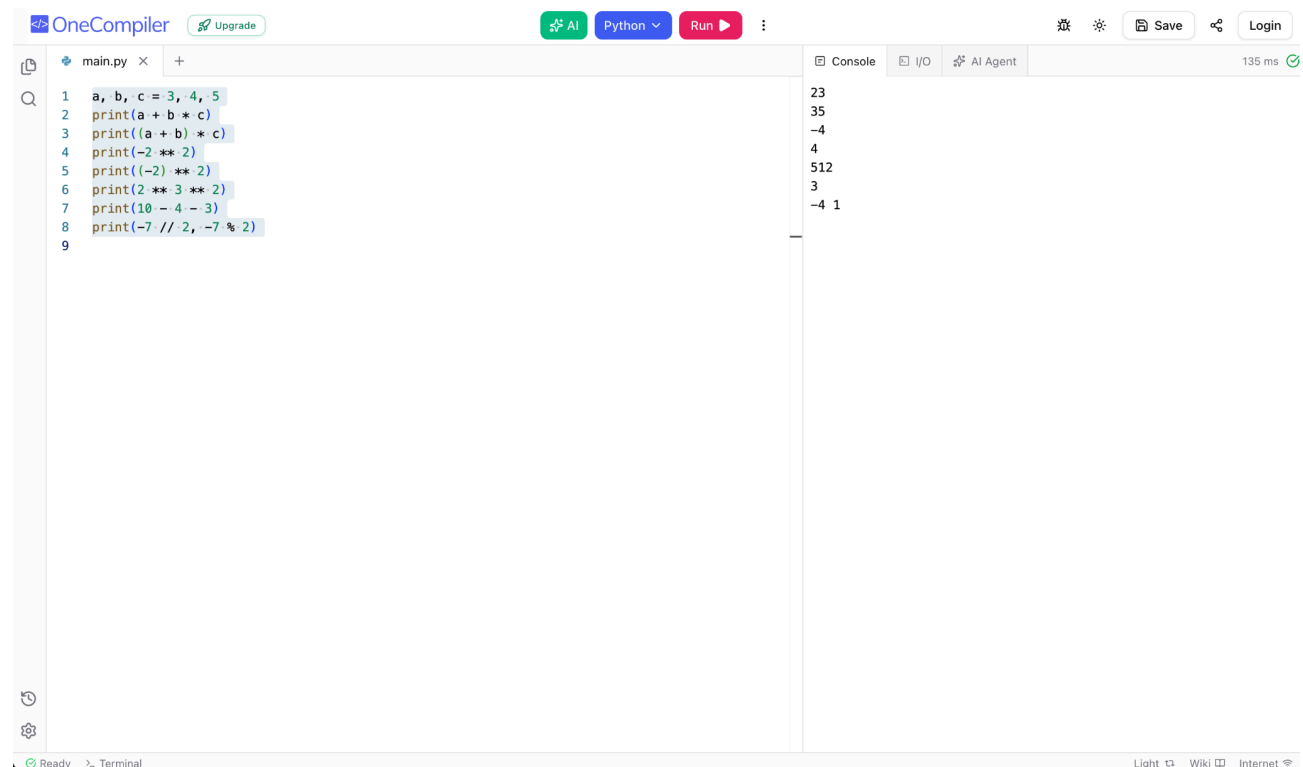
2.1 Precedência e associatividade

Precedência define qual operador é avaliado primeiro (`*` antes de `+`). Associatividade decide a ordem quando dois operadores têm a mesma precedência: `10 - 4 - 3` é `(10 - 4) - 3`. Quase tudo associa à esquerda, mas a exponenciação `**` associa à direita em Python, JavaScript, Ruby e Fortran, então `2 ** 3 ** 2` é `2 ** 9`, ou seja, 512.

Precedência é convenção de projeto, não lei. APL avalia sempre da direita para a esquerda (`3 × 4 + 5` dá 27) e Lisp nem precisa de precedência, porque usa notação prefixa com parênteses, como em `(+ a (* b c))`.

Pegadinhas: em Python, `-2 ** 2` dá `-4`, porque `**` tem precedência maior que o menos unário. Em JavaScript essa mesma expressão é erro de sintaxe (obriga parênteses). No Excel, `=-2^2` dá 4. A divisão também muda: Java e C truncam em direção a zero (`-7 / 2` é `-3`), enquanto Python arredonda para baixo (`-7 // 2` é `-4`). Na dúvida, usar parênteses.

Print 1: Precedência e associatividade (Python)



```
1 a, b, c = 3, 4, 5
2 print(a + b * c)
3 print((a + b) * c)
4 print(-2 ** 2)
5 print((-2) ** 2)
6 print(2 ** 3 ** 2)
7 print(10 - 4 - 3)
8 print(-7 // 2, -7 % 2)
9
```

Console output:

```
23
35
-4
4
512
3
-4 1
```

O que observar: a saída esperada é 23, 35, -4, 4, 512, 3 e -4 1. O `-2 ** 2` dar -4 (e não 4) é o ponto principal.

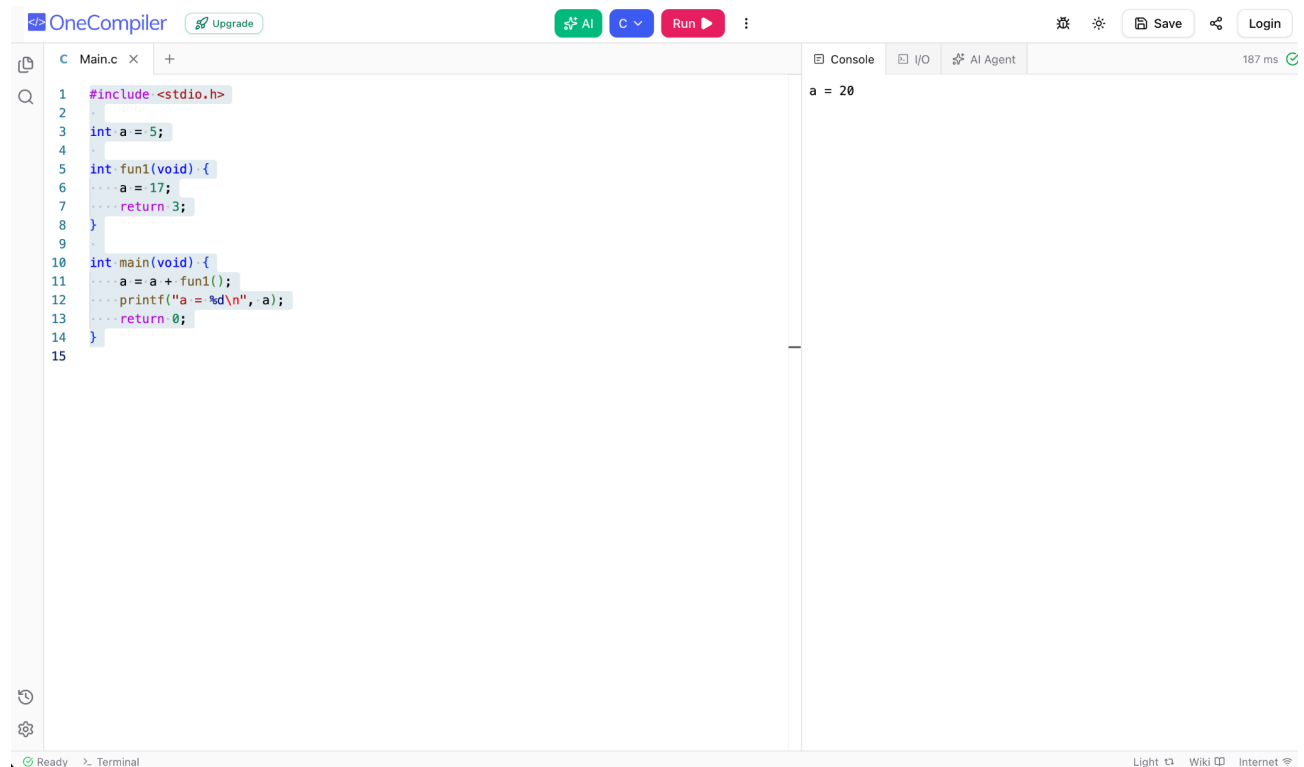
2.2 Ordem dos operandos e efeitos colaterais

Efeito colateral funcional é quando uma função altera uma variável global ou um parâmetro. Quando isso acontece, a ordem de avaliação dos operandos muda o resultado. No exemplo do livro, `a = a + fun1()`, com `a = 5` e `fun1` mudando `a` para 17 e devolvendo 3, o resultado é 8 se `a` for lido antes da chamada e 20 se a chamada acontecer primeiro.

Nos testes do professor, Java dá 8, porque a linguagem garante a avaliação da esquerda para a direita. C (gcc) e Go dão 20, porque o padrão de C não define a ordem e Go não define quando a variável é lida em relação à chamada.

Transparência referencial: sem efeitos colaterais, a ordem não importa e duas expressões com o mesmo valor podem ser trocadas (`f(x) + f(x)` vira `2 * f(x)`). Funções puras garantem isso.

Print 2: Ordem de avaliação e efeito colateral (C)



The screenshot shows the OneCompiler interface. The code editor on the left contains the following C code:

```
1 #include <stdio.h>
2
3 int a = 5;
4
5 int fun1(void) {
6     a = 17;
7     return 3;
8 }
9
10 int main(void) {
11     a = a + fun1();
12     printf("a = %d\n", a);
13     return 0;
14 }
15
```

The console on the right shows the output: `a = 20`. The status bar at the bottom indicates 'Ready' and 'Terminal'.

O que observar: no gcc a saída é `a = 20`. Em Java, o mesmo programa (classe `Main`, com `static int a = 5` e a mesma `fun1`) dá `a = 8`.

2.3 Sobrecarga de operadores

Sobrecarga é usar o mesmo símbolo com mais de um significado. Em Java, o `+` soma inteiros, soma reais e concatena texto: `'a' + 1` dá 98 (o char vira int) e `"" + 'a' + 1` dá `a1`. C++, C#, Python, Rust e Kotlin deixam o programador definir a sobrecarga (Python com `__add__`, Rust com a trait `Add`). Java não deixa, e por isso `BigDecimal` usa `x.add(y)` em vez de `x + y`. A sobrecarga melhora a leitura (`v1 + v2`), mas pode esconder custo e comportamento.

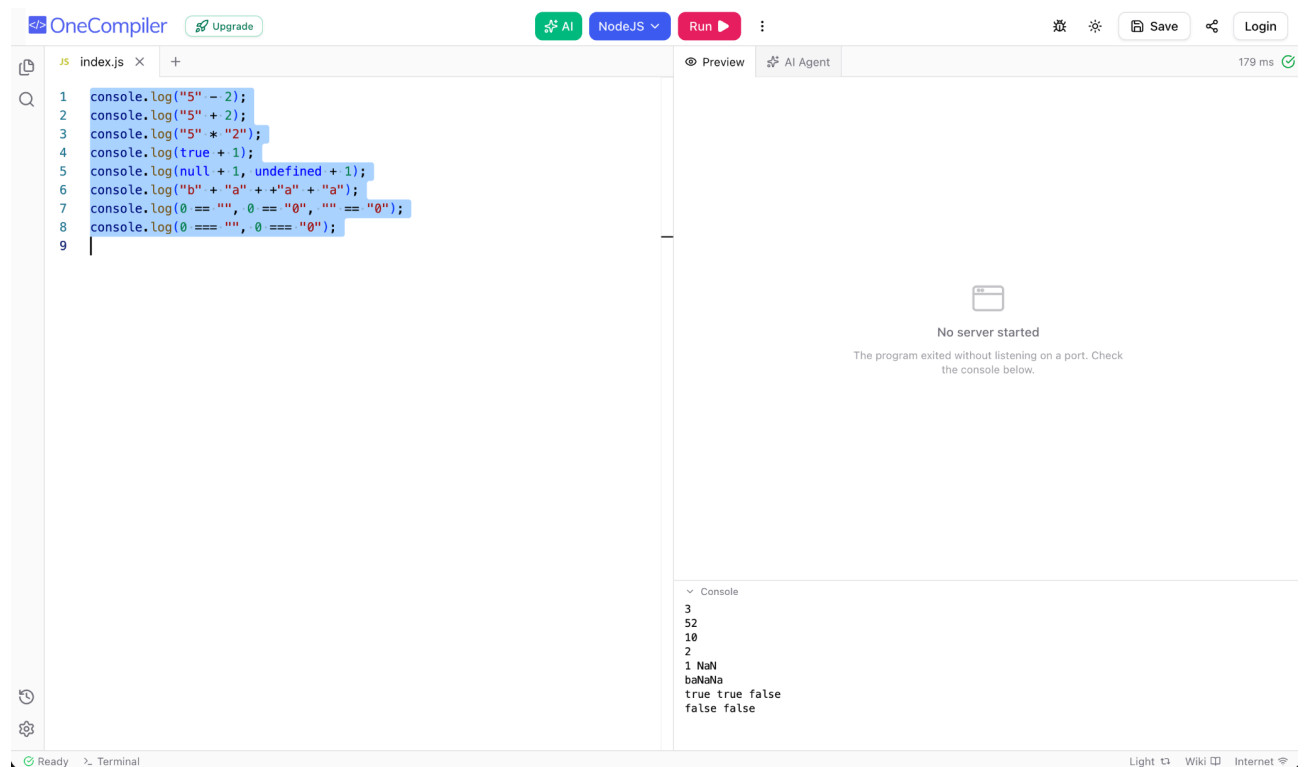
2.4 Conversões de tipo

Alargamento converte para um tipo maior (int para double). Costuma ser seguro, mas pode perder precisão: `123456789` convertido para `float` e de volta para `int` vira `123456792`. Estreitamento vai para um tipo menor e pode perder informação: `(int) 3.99` dá 3 e `(byte) 200` dá -56. Coerção é a conversão implícita feita pelo compilador, e `cast` é a conversão explícita pedida pelo programador.

A coerção pode esconder erro de digitação. Por isso Go, Rust, ML e F# não fazem conversão implícita entre tipos numéricos.

JavaScript e PHP convertem de forma bem agressiva. Em JS: `"5" - 2` dá 3, `"5" + 2` dá `"52"`, `"b" + "a"` dá `ba`, `"a" + "a"` dá `baNaN`, e `0 == ""` e `0 == "0"` são verdadeiros, mas `"" == "0"` é falso. Em PHP, `"0e123" == "0e456"` é verdadeiro. **Regra prática:** em JavaScript e PHP, usar `===` e `!==`, que não fazem coerção.

Print 3: Coerção implícita (JavaScript)



The screenshot shows the OneCompiler interface. The code editor on the left contains the following JavaScript code:

```
1 console.log("5" - 2);
2 console.log("5" + 2);
3 console.log("5" * "2");
4 console.log(true + 1);
5 console.log(null + 1, undefined + 1);
6 console.log("b" + "a" + "a" + "a");
7 console.log(0 == "", 0 == "0", "" == "0");
8 console.log(0 === "", 0 === "0");
9
```

The right panel shows a message: "No server started. The program exited without listening on a port. Check the console below." Below this, the console output is displayed:

```
3
52
10
2
1 NaN
baNaNa
true true false
false false
```

O que observar: a saída esperada é 3, 52, 10, 2, 1 NaN, baNaNa, true true false e false false. O == dando resultados que não fecham entre si (não transitivo) é o principal.

2.5 Expressões relacionais, booleanas e curto-circuito

Em C não existe booleano de verdade: 0 é falso e qualquer outro valor é verdadeiro. Por isso `3 > 2 > 1` dá 0 em C, já que `3 > 2` vale 1 e `1 > 1` é falso. Em Python a comparação é encadeada, então `3 > 2 > 1` é True. Python e JavaScript aceitam qualquer valor em um `if` (0, vazio e nulo contam como falso), enquanto Java, C#, Go e Rust exigem um booleano.

Curto-circuito é quando o resultado é decidido sem avaliar todos os operandos. É isso que protege o `while (i < lista.length && lista[i] != chave)` de acessar uma posição que não existe. Em Java, `&&` faz curto-circuito e `&` não. Cuidado com efeito colateral no segundo operando, que pode não acontecer.

Em Python, `and` e `or` devolvem um dos operandos: `0 or "padrão"` dá "padrão". Em JavaScript, `quantidade || 10` perde o valor 0, e o `??` resolve, porque só troca `null` e `undefined`.

3. Atribuição

Em C e JavaScript a atribuição é uma expressão, ou seja, devolve o valor atribuído. Isso permite `sum = count = 0` e `while ((ch = getchar()) != EOF)`. O custo é que `if (x = y)` compila e atribui em vez de comparar. Java e C# evitam o problema porque o `if` exige booleano. Em Python, Go e Rust a atribuição não é expressão (Python só tem a "morsa" `:=`).

Outras formas: atribuição composta (`a += b`), unária (`count++`) e múltipla (`a, b = b, a`, em Python e Go). Em linguagens funcionais e em Rust, `let` dá nome a um valor que não muda, e `let x = x + 1` cria uma nova variável (sombreamento).

Atribuição de modo misto: C converte em qualquer direção e em silêncio (`int i = 3.7` vira 3 e `char c = 300` vira 44). Java só aceita alargamento sem cast, mas `i += 3.7` compila, porque o `+=` inclui um cast escondido. Armadilha clássica: `double taxa = acertos / total * 100;` com inteiros dá 0.0, porque a divisão inteira acontece antes do alargamento para double. Essa é a questão 6 do exercício.

4. Estruturas de controle

4.1 Quantas estruturas são necessárias?

Böhm e Jacopini (1966) provaram que sequência, seleção e laço lógico bastam para escrever qualquer algoritmo, e o `goto` não é necessário. As outras estruturas (`for`, `switch`, `do`, `foreach`) existem por facilidade de escrita. O limite é a legibilidade: quanto maior a linguagem, menos do código dos outros a pessoa reconhece.

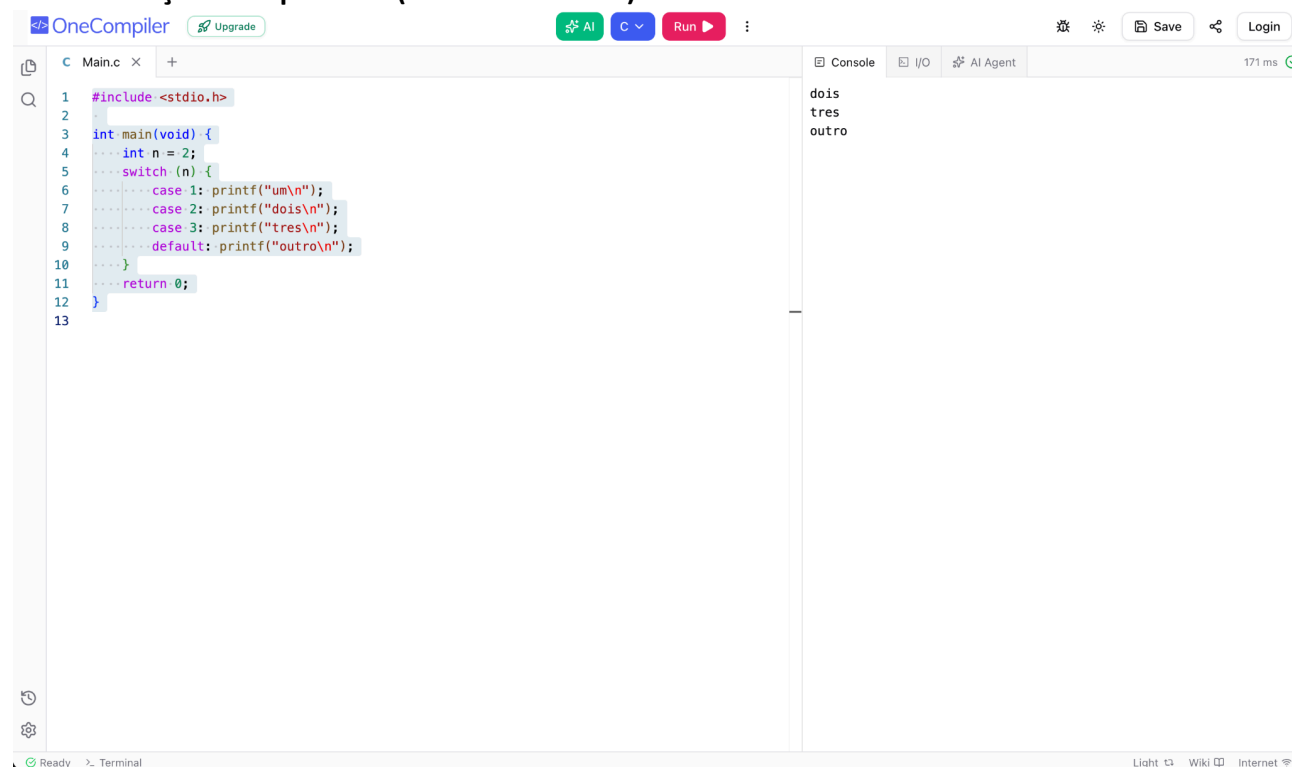
4.2 Seleção de dois caminhos: o "else pendente"

Em Java, C, C++ e C#, o `else` pertence ao `if` mais próximo que ainda não tem `else`, e a endentação é ignorada (só Python e F# levam a endentação em conta). No exemplo `if (sum == 0) if (count == 0) result = 0; else result = 1;`, o `else` é do `if` interno, e `result` acaba valendo 1 mesmo que a endentação sugira outra coisa. Chaves obrigatórias (Perl, Go e Rust) eliminam o problema. Em Rust o `if` é uma expressão que devolve valor, e em Python existe a forma `x if c else y`.

4.3 Seleção múltipla: `switch` de C, `switch` com `->` de Java, `switch` de Go e `match` de Rust

Para comparar, o mesmo exemplo foi escrito nas quatro linguagens, com `n = 2`. A pergunta é o que cada uma imprime. Em C, sem `break`, o controle "cai" nos casos seguintes.

Print 4: Seleção múltipla em C (switch sem break)

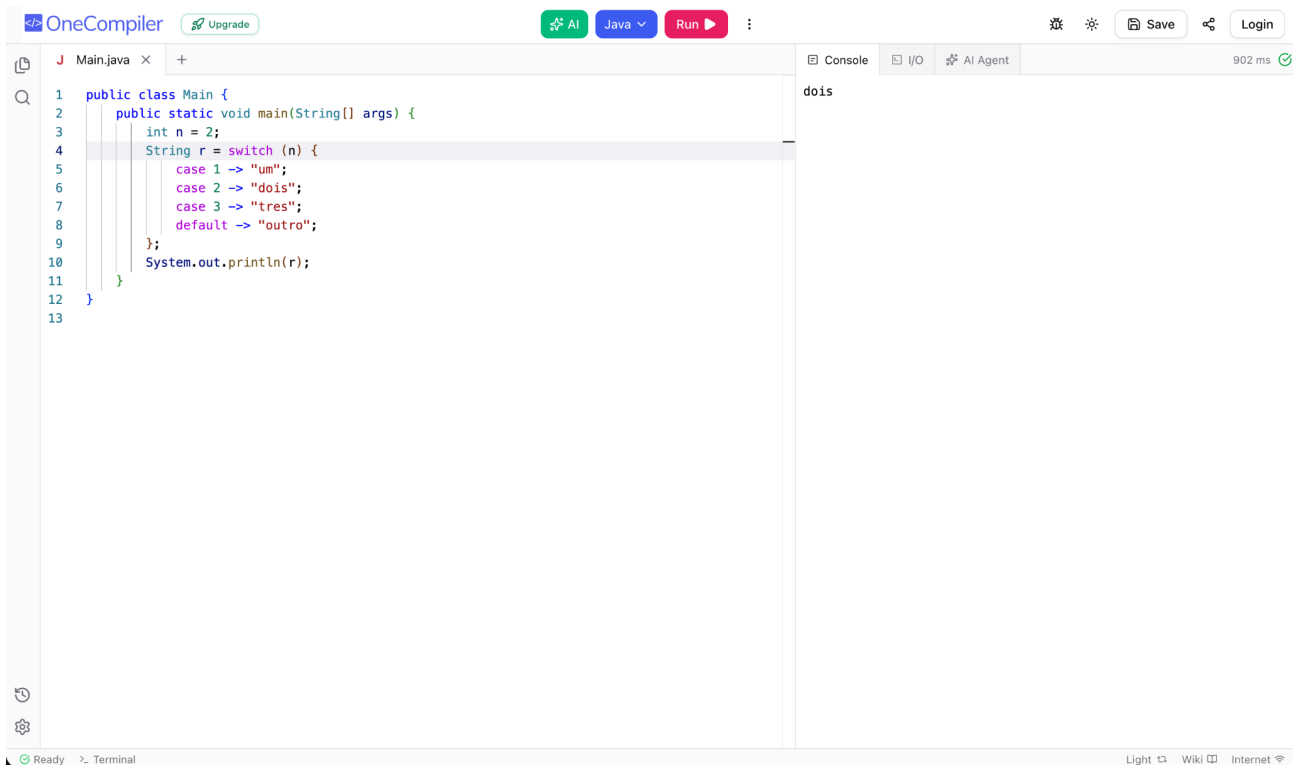


```
1 #include <stdio.h>
2
3 int main(void) {
4     int n = 2;
5     switch (n) {
6         case 1: printf("um\n");
7         case 2: printf("dois\n");
8         case 3: printf("tres\n");
9         default: printf("outro\n");
10    }
11    return 0;
12 }
13
```

Console output: dois, tres, outro

O que observar: imprime `dois`, `tres` e `outro`, cada um em uma linha. O controle caiu do caso 2 até o fim.

Print 5: Seleção múltipla em Java (switch com ->)



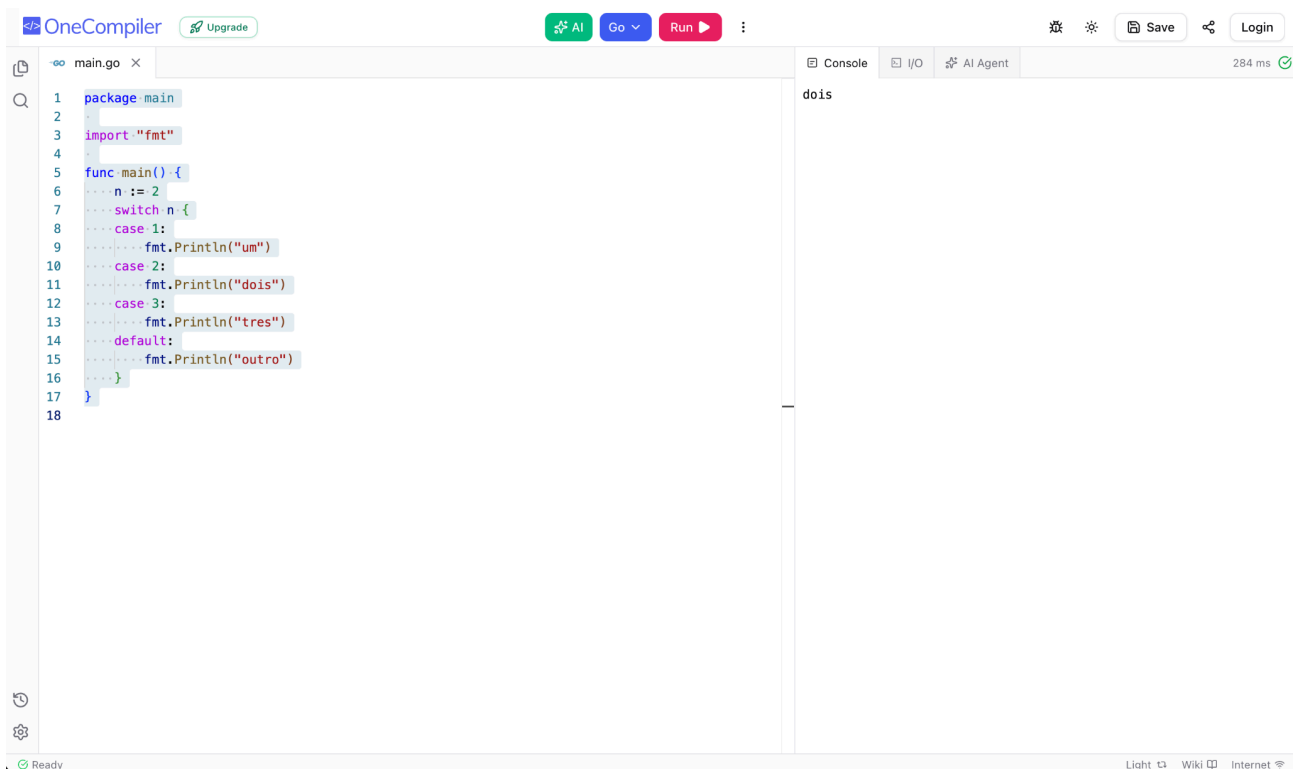
The screenshot shows the OneCompiler Java IDE. The code in Main.java is as follows:

```
1 public class Main {
2     public static void main(String[] args) {
3         int n = 2;
4         String r = switch (n) {
5             case 1 -> "um";
6             case 2 -> "dois";
7             case 3 -> "tres";
8             default -> "outro";
9         };
10        System.out.println(r);
11    }
12 }
13
```

The console output on the right is "dois". The execution time is 902 ms.

O que observar: imprime só dois. Sem `break` e sem `case`, e o `switch` virou uma expressão que devolve valor. Para `int` o `default` é obrigatório.

Print 6: Seleção múltipla em Go



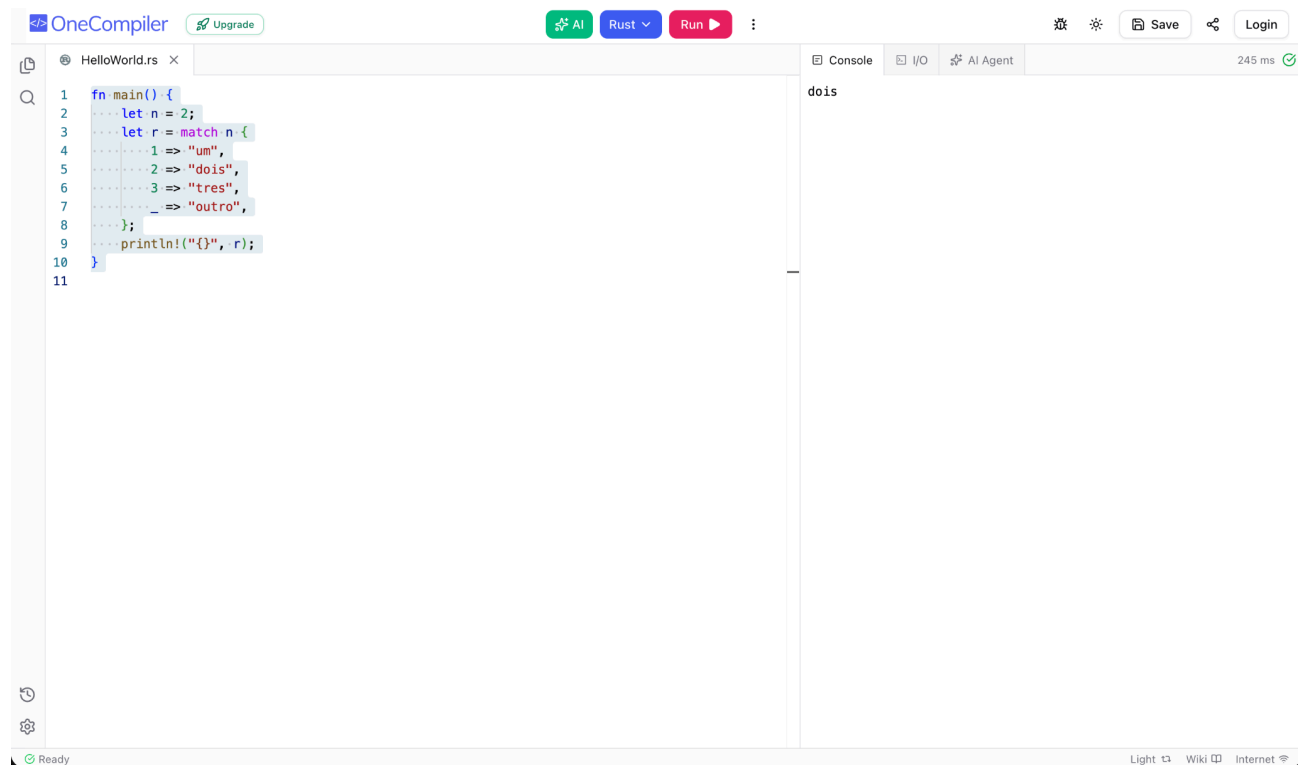
The screenshot shows the OneCompiler Go IDE. The code in main.go is as follows:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     n := 2
7     switch n {
8     case 1:
9         fmt.Println("um")
10    case 2:
11        fmt.Println("dois")
12    case 3:
13        fmt.Println("tres")
14    default:
15        fmt.Println("outro")
16    }
17 }
18
```

The console output on the right is "dois". The execution time is 284 ms.

O que observar: imprime só dois. Cada caso termina sozinho, e cair no próximo exigiria a palavra `fallthrough`.

Print 7: Seleção múltipla em Rust (match)



```
1 fn main() {
2     let n = 2;
3     let r = match n {
4         1 => "um",
5         2 => "dois",
6         3 => "tres",
7         _ => "outro",
8     };
9     println!("{}", r);
10 }
11
```

Console: dois

O que observar: imprime só `dois`. Se a linha do `_` for apagada, o código não compila, porque o `match` precisa cobrir todos os casos.

Comparação das quatro:

C: o controle cai nos casos seguintes quando não há `break`, o `default` não é obrigatório e é fácil esquecer o `break`, o que gera bug.

Java (->): não cai, é exaustivo para `enum` (para `int` o `default` é obrigatório), é legível e o `switch` vira uma expressão.

Go: não cai (só com `fallthrough`), não exige `default` e dispensa o `break`.

Rust (`match`): não cai, cobrir todos os casos é obrigatório (faltar caso é erro de compilação) e aceita faixas e padrões.

Conclusão: C é a única em que o padrão é cair, e o livro chama esse `switch` de "relativamente primitivo" (o `break` é um goto restrito). As outras três evitam essa classe de bug, e o `match` do Rust ainda força cobrir todos os casos.

4.4 Laços

No `for` de C as três expressões são opcionais e a variável pode ser alterada no corpo: `for (int i = 0; i < 10; i++) { printf("%d ", i); i += 2; }` imprime 0 3 6 9. O `for` de Python percorre um iterável, então mudar `i` dentro do corpo não afeta nada (imprime de 0 a 9).

Nos laços lógicos, o teste pode ser antes (`while`) ou depois (`do-while`, que executa ao menos uma vez). Java e Perl têm `break` rotulado, que sai de dois laços de uma vez. Em JavaScript, `for...in` percorre as chaves (que são strings) e `for...of` percorre os valores: somar com `for...in` dá "0012".

Caso Zune: com `dias == 366` em um ano bissexto, nenhum ramo do `while` alterava `dias`, e o laço nunca terminava. A pergunta certa é qual invariante o laço deveria garantir: a cada volta, `dias` precisa diminuir.

4.5 goto e comandos protegidos

O `goto` é a sentença de controle mais poderosa, e por isso a mais perigosa: deixa o programa difícil de ler e de manter (Dijkstra, 1968, "Go To Statement Considered Harmful"). Java não tem `goto`, enquanto C, C++, C# e Go têm. `break`, `continue` e `return` são `gotos` restritos e melhoram a legibilidade.

No caso do `goto fail`, a segunda linha `goto fail;` não pertencia ao `if`, porque não havia chaves. Com isso a verificação da assinatura nunca rodava e `err` continuava valendo 0, que significa "válido". Chaves obrigatórias (Go e Rust) ou o aviso de endentação enganosa do `gcc -Wall` teriam evitado o problema.

Comandos protegidos de Dijkstra: no `if ... fi` e no `do ... od`, todas as guardas são avaliadas. Se mais de uma for verdadeira, uma é escolhida de forma não determinística. Se nenhuma for, é erro no `if` e fim do laço no `do`. São a base do CSP de Hoare e do `select` de Go, que escolhe ao acaso quando vários canais estão prontos. Em Haskell as guardas são testadas em ordem, então são determinísticas.

5. Exercício "Encontre o bug"

Para cada trecho: (a) qual é a saída, (b) qual regra causa o problema e (c) como corrigir. As respostas abaixo foram conferidas rodando o código.

Print 8: Questão 1 (JavaScript)

The screenshot shows the OneCompiler JavaScript editor. The code in `index.js` is:

```
1 const precos = [10, 20, 30];
2 let total = 0;
3 for (const p in precos) total += p;
4 console.log("Total: " + total);
5
```

The right panel shows the "Preview" tab with a message: "No server started. The program exited without listening on a port. Check the console below." The "Console" tab shows the output: "Total: 0012".

(a) Saída: Total: 0012.

(b) Regra: `for...in` percorre as chaves (as strings "0", "1", "2"), e o `+` concatena.

(c) Correção: usar `for...of`, ou `precos.reduce((a, b) => a + b, 0)`.

Print 9: Questão 2 (Python)

The screenshot shows the OneCompiler Python IDE. The code in `main.py` is as follows:

```
1 def desconto(preco, d=None):
2     d = d or 10 # padrão: 10%
3     return preco * (100 - d) / 100
4
5 print(desconto(200, 0))
6
```

The console output is `180.0`, and the execution time is 123 ms.

- (a) Saída: 180.0, mas o esperado era 200.0 (desconto zero).
- (b) Regra: o `d or 10` trata o 0 como falso, e o `or` devolve o segundo operando.
- (c) Correção: `if d is None: d = 10`. Em JavaScript, o `??` resolveria.

Print 10: Questão 3 (Java)

The screenshot shows the OneCompiler Java IDE. The code in `Main.java` is as follows:

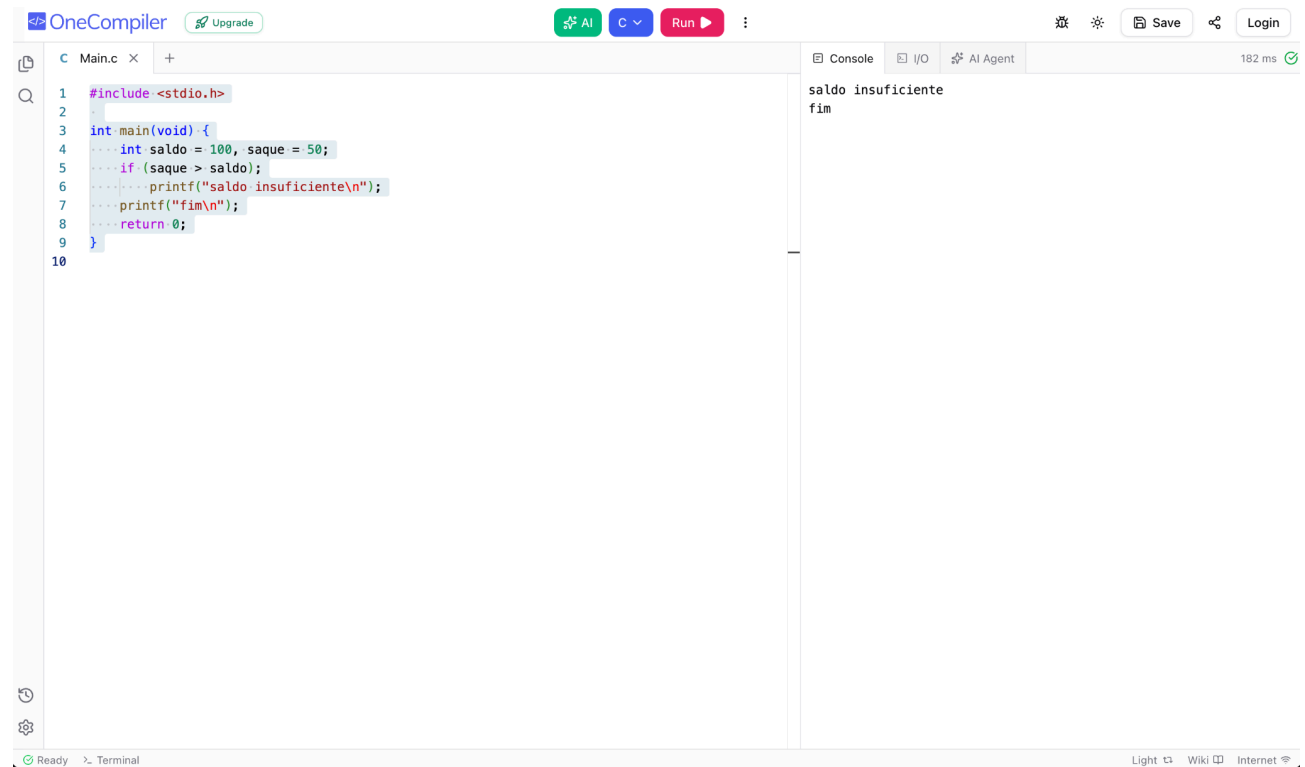
```
1 public class Main {
2     public static void main(String[] args) {
3         int dia = 2;
4         String nome = "";
5         switch (dia) {
6             case 1: nome = "domingo";
7             case 2: nome = "segunda";
8             case 3: nome = "terça";
9             default: nome = "inválido";
10        }
11        System.out.println(nome);
12    }
13 }
14
```

The console output is `inválido`, and the execution time is 873 ms.

- (a) Saída: inválido.
- (b) Regra: o `switch` sem `break` cai do `case 2` até o `default`.

(c) Correção: colocar `break` em cada caso, ou usar o `switch` com `->` (Java 14+).

Print 11: Questão 4 (C)



The screenshot shows the OneCompiler IDE interface. The main editor displays a C program in a file named `Main.c`. The code is as follows:

```
1 #include <stdio.h>
2
3 int main(void) {
4     int saldo = 100, saque = 50;
5     if (saque > saldo);
6     printf("saldo insuficiente\n");
7     printf("fim\n");
8     return 0;
9 }
10
```

The right-hand pane shows the output of the program in the Console tab:

```
saldo insuficiente
fim
```

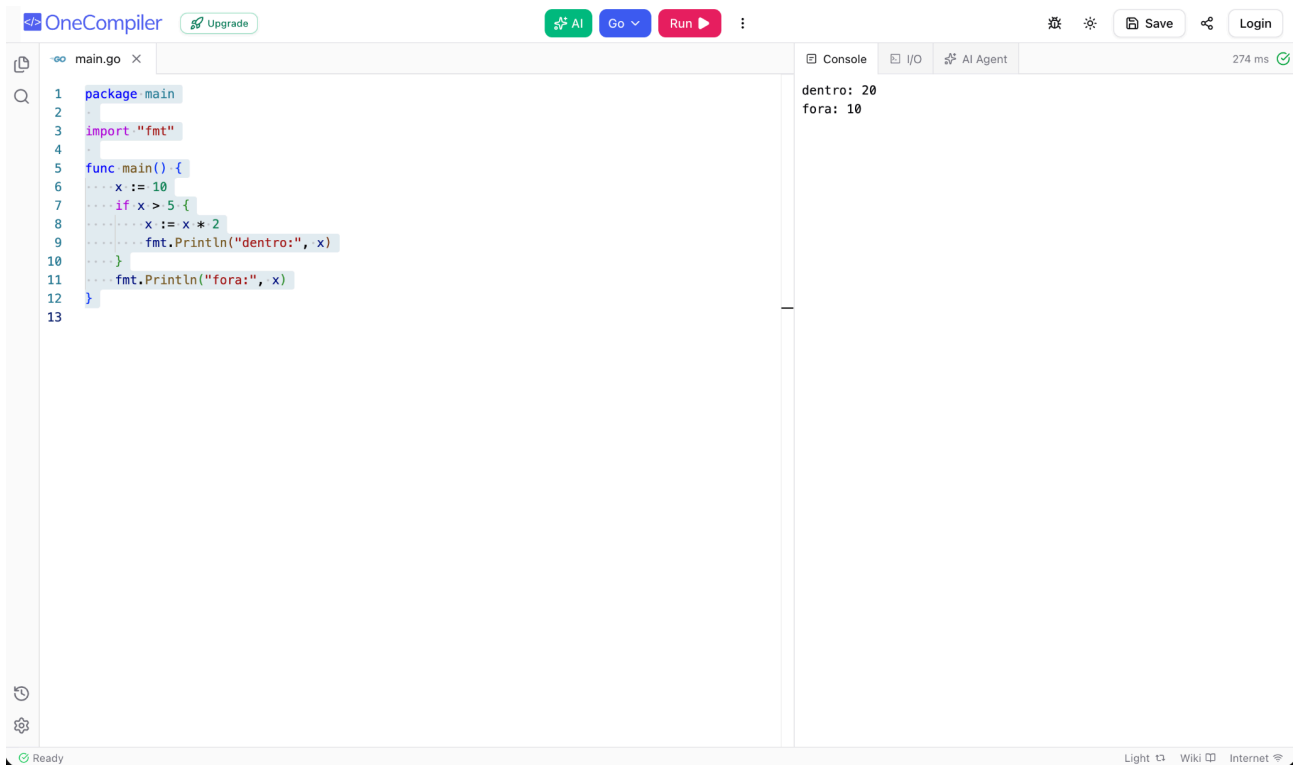
The top of the IDE includes a toolbar with buttons for AI, C, Run, Save, and Login. The bottom status bar indicates the compiler is 'Ready' and shows a 'Terminal' tab.

(a) Saída: `saldo insuficiente` e `fim`, mesmo com o saque menor que o saldo.

(b) Regra: o `;` logo depois do `if` é uma sentença vazia, então o `printf` fica fora do `if`.

(c) Correção: usar chaves sempre. O `gcc -Wall` avisa (`-Wempty-body`).

Print 12: Questão 5 (Go)



The screenshot shows the OneCompiler Go editor interface. The code editor on the left contains the following Go code:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     x := 10
7     if x > 5 {
8         x := x * 2
9         fmt.Println("dentro:", x)
10    }
11    fmt.Println("fora:", x)
12 }
13
```

The console on the right shows the output of the program:

```
dentro: 20
fora: 10
```

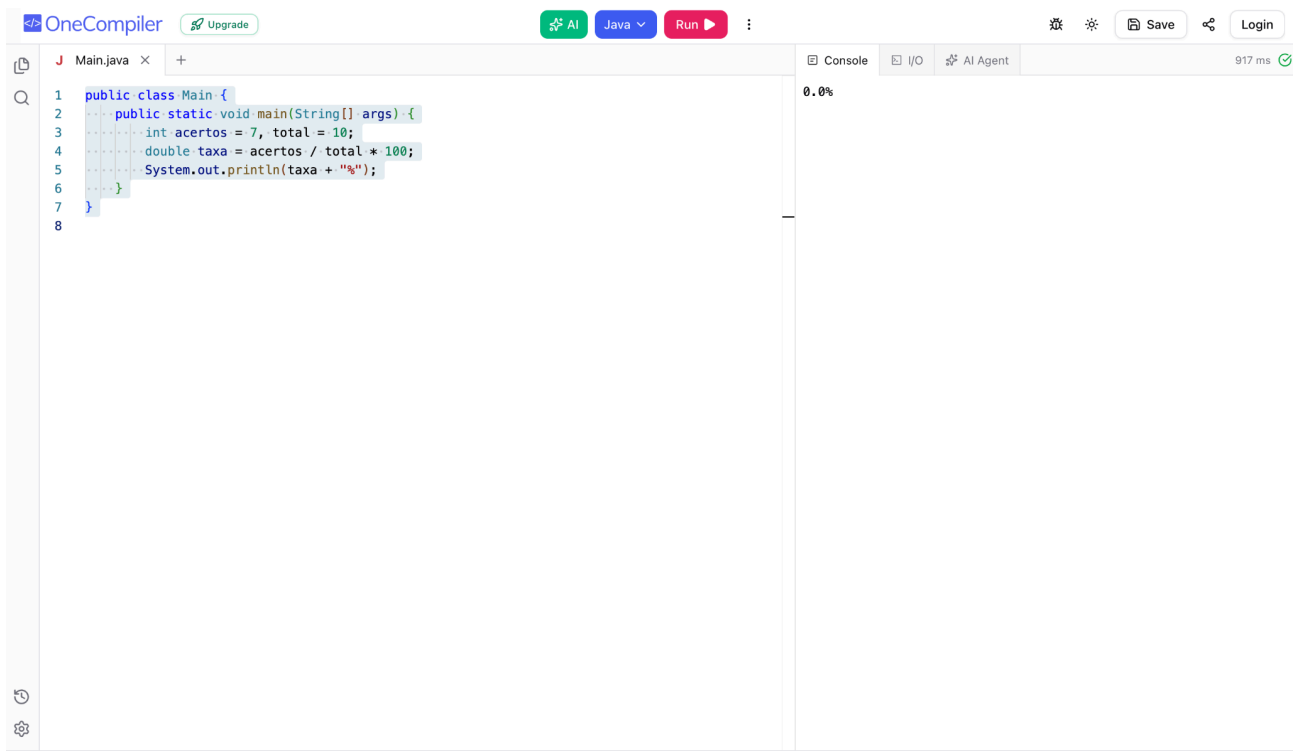
The top bar of the editor includes the OneCompiler logo, an Upgrade button, and buttons for AI, Go, Run, Save, and Login. The bottom status bar shows 'Ready' and links to Light, Wiki, and Internet.

(a) Saída: dentro: 20 e fora: 10.

(b) Regra: o `x :=` dentro do `if` cria uma nova variável `x` (sombreamento) em vez de alterar a de fora.

(c) Correção: usar `x = x * 2`.

Print 13: Questão 6 (Java)



The screenshot shows the OneCompiler Java editor interface. The code editor on the left contains the following Java code:

```
1 public class Main {
2     public static void main(String[] args) {
3         int acertos = 7, total = 10;
4         double taxa = acertos / total * 100;
5         System.out.println(taxa + "%");
6     }
7 }
8
```

The console on the right shows the output of the program:

```
0.0%
```

The top bar of the editor includes the OneCompiler logo, an Upgrade button, and buttons for AI, Java, Run, Save, and Login. The bottom status bar shows 'Ready' and links to Light, Wiki, and Internet.

(a) Saída: 0.0%.

(b) Regra: $7 / 10$ é divisão inteira (dá 0) e só depois o resultado é alargado para double (atribuição de modo misto).

(c) Correção: `acertos * 100.0 / total` ou `(double) acertos / total * 100`, que dão 70.0%.

6. Autopesquisa

Expressões em Ruby: os operadores são métodos ($1 + 2$ é o mesmo que `1.+(2)`) e podem ser redefinidos. O `**` associa à direita, como em Python. Os iteradores usam blocos, como em `3.times { |i| print i }`, que imprime 0 1 2.

Expressões em Lisp: a notação é prefixa e os parênteses dizem a ordem, então não existe precedência. `(+ a (* b c))` é `a + b * c`, e `(+ 1 2 3 4 5)` mostra um operador com vários operandos. O `if` também é expressão: `(if (> a 0) a (* 2 a))`.

Expressões condicionais: escolhem um valor dentro de uma expressão. Em C e Java é o operador ternário `c ? a : b`, em Python é `x if c else y`, e em Rust e F# o próprio `if` devolve valor.

Erros em expressões: pelo que consta no livro (seção 7.4.3, p. 313), são erros que surgem por limitações da aritmética do computador, como estouro de valor (overflow) e divisão por zero. A linguagem pode tratar com exceções ou deixar passar sem aviso.

Tabelas de desvio: em um `switch` com muitos casos, o compilador pode gerar uma tabela de desvios (acesso direto ao caso) ou uma busca binária, em vez de testar um por um. Por isso um `switch` grande pode ser mais rápido que uma cadeia de `if`.

Laços em linguagens funcionais: como não há variável que muda, a contagem é feita por recursão.

Iteradores e blocos de Ruby: em vez de um `for` com contador, o objeto recebe um bloco de código e o executa para cada elemento (`each`, `times`, `select`). É a mesma ideia do `forEach` de JavaScript.

Leitura (Dijkstra, 1968): é o artigo curto que defende que o `goto` torna o programa difícil de entender e de provar correto. Knuth, em 1974, defendeu que alguns usos do `goto` são aceitáveis, como saltar para um bloco comum de limpeza ao tratar erros em C.

7. Síntese

Precedência: a armadilha está em `-2 ** 2` e `1 + 2 + "3"`. Ajudam os parênteses, e o JS obriga a escrever `(-2) ** 2`.

Operandos: `a + f()` com efeito colateral dá resultados diferentes em C e Go. Java fixa a ordem, e funções puras resolvem o problema.

Conversões: a coerção silenciosa (C, JS, PHP) esconde erros. Go e Rust exigem conversão explícita, e em JS vale usar `===`.

Booleanos: 0, "" e [] contam como falsos, e `a > b > c` em C engana. Java, Go e Rust só aceitam booleano no `if`, e existe o `??`.

Atribuição: `if (x = y)` compila em C. Em Python, Go e Rust a atribuição não é expressão.

Seleção: `switch` sem `break` e `else` pendente. Ajudam o `->`, o `match` exaustivo e as chaves obrigatórias.

Laços: `for...in` no JS e laço sem progresso. Ajudam o `for...of`, os iteradores e o `break` rotulado.

Desvio: `goto` sem restrição. `break`, `continue` e `return` são `gotos` restritos e mais legíveis.

As linguagens mais novas (Go, Rust, Kotlin, Swift) aprenderam com os erros de C: exigem booleano no `if`, chaves obrigatórias, conversões explícitas, `switch` ou `match` sem `case` e atribuição como sentença. Cada restrição elimina uma classe inteira de bugs, trocando um pouco de facilidade de escrita por confiabilidade, que é o trade-off do Cap. 1.

Referências: SEBESTA, R. W. Conceitos de Linguagens de Programação, 11ª ed., Caps. 7 e 8 (p. 299-361). Slides e exemplos da Aula 07.